

## REPORT WEEK 3

In the week 2 I created the fir filter and verified it using verilog testbench.

The target of this week is to code and simulate uart rx and tx modules and integrate rx to fir to tx with proper handshaking.

So first i planned to create the uart implementation.

For that i decided to create 3 modules

1. baud rate generator
2. rx
3. tx
4. uart top module

### Baud Rate Generator

The fpga i am having have a internal clock signal of 12mhz. I need to create a uart communication of baud rate 115200. so that is it would send 115200 bits per second. Since while recieving a bit we must sample it multiple times in order to verify if it is a glitch or is it actually a bit. For that i decided to use take 5 samples and check for the majority vote case. Since the 5 samples i should take should be on the center of the bits recievign i planned to sample the bits 16 times and take the center 5 samples and check for majority votes.

Since i need 16 samples per bit i need to generate another clock of  $115200 \times 16$ . so i can sample every bit 16 times.

What i did is that we know number of cycles=freq x time. I have a clock freq 12mhz and i need another clock of  $115200 \times 16$ . So  $t=1/f$ .  $12\text{mhz}/(115200 \times 16)$  give number of clock cycles. Which is 6.5.

So i 6.5 clock cycles to generate that freq. But 6.5 is not possible i need a integer. So i created a accumulator register and on every posedge of clk it would add  $115200 \times 16$  to it and then if the accumulator value is greater than  $12000000 - 115200 \times 16$  that is it would overflow if another posedge occurs then i would assign accumulator register to be the current value +  $115200 \times 16 - 12000000$ . that is essentially like taking the overflow value and resetting the overflow value to a new reg. And producing a tick when the overflow occuring. This would maintain the shift occuring by making it 6 clock cycle some time and 7 clock cycle some time maintiainging the baudratex16 pulses.

### Verilog code

```
module baud_generator(
input clk,
input rstn,
output reg tick
);
localparam system_clk=12_000_000;
localparam baud_rate=115200;
localparam rx_needed=(baud_rate*16);
reg [23:0] acc_rx=24'd0;
always @(posedge clk) begin
if (!rstn) begin
tick <= 1'b0;
acc_rx <= 24'd0;
end
```

```

else begin
    tick <= 1'b0;
    if(acc_rx>=system_clk-rx_needed) begin
        tick<=1'b1;
        acc_rx<=acc_rx+rx_needed-system_clk;
    end
    else begin
        acc_rx<=acc_rx+rx_needed;
    end
end
end
endmodule

```

I decided to verify it using python cocotb verification test bench  
The plan was that to see if the tick is occuing for 6 cycles and 7 cyles

```

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge, ReadOnly

```

```

@cocotb.test()
async def test_baud_generator(dut):
    cocotb.start_soon(Clock(dut.clk, 83.33, units="ns").start())
    dut.rstn.value=0
    await RisingEdge(dut.clk)
    dut.rstn.value=1
    for i in range(3):
        count=0
        while True:
            await RisingEdge(dut.clk)
            await ReadOnly()
            count=count+1
            if(dut.tick.value==1):
                break

```

```

    dut._log.info(f"tick {i} fired with count {count}")

```

```

/home/kevin/Desktop/uart_streamed_fir_filter/.venv/lib/python3.13/site-packages/cocotb_tools/makefiles/simulators/Makefile.icarus:68: Using MODULE is deprecated, p
tead.
rm -f results.xml
COCOTB_TEST_MODULES=tb_baud_generator COCOTB_TESTCASE= COCOTB_TOPLEVEL=baud_generator TOPLEVEL_LANG=verilog \
/usr/bin/vvp -m /home/kevin/Desktop/uart_streamed_fir_filter/.venv/lib/python3.13/site-packages/cocotb/libs/libcocotbvpi_icarus.so sim_build/sim.vvp
--ns INFO gpi ../gpi/GpiCommon.cpp:242 in gpi_load_users Loaded entry library: '/usr/lib/x86_64-linux-g
--ns INFO gpi ../gpi/GpiCommon.cpp:233 in gpi_load_users Running entry func 'initialize' from loaded li
treamed_fir_filter/.venv/lib/python3.13/site-packages/cocotb/simulator.cpython-313-x86_64-linux-gnu.so'
--ns INFO pygpi ../lib/pygpi/embed.cpp:114 in initialize Using Python 3.13.8 interpreter at /home/kevin
/.venv/bin/python3
--ns INFO gpi ../gpi/GpiCommon.cpp:74 in gpi_print_registered_impl GPI: VPI support registered
0.00ns INFO cocotb.initialize Seeding Python random module with 1790142997
0.00ns INFO cocotb.initialize Initialized cocotb v2.1.0 from /home/kevin/Desktop/uart_streamed_fir_filter/.venv/lib/python3.13/site-packa
0.00ns INFO cocotb.initialize Running on Icarus Verilog version 12.0 (stable)
0.00ns INFO cocotb.regression Running tests
0.00ns INFO cocotb.regression running tb_baud_generator.test_baud_generator (1/1)
0.00ns WARNING py_warnings /home/kevin/Desktop/uart_streamed_fir_filter/verilog/uart/baud_generator/tb_baud_generator.py:7: Deprecati
s been renamed to 'unit'.
cocotb.start_soon(Clock(dut.clk, 83.33, units="ns").start())

583.31ns INFO cocotb.baud_generator tick 0 fired with count 7
1166.62ns INFO cocotb.baud_generator tick 1 fired with count 7
1666.60ns INFO cocotb.baud_generator tick 2 fired with count 6
1666.60ns INFO cocotb.regression tb_baud_generator.test_baud_generator passed
1666.60ns INFO cocotb.regression
*****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** tb_baud_generator.test_baud_generator PASS 1666.60 0.01 324246.73 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0 1666.60 0.01 288313.21 **
*****

```

## RX module

the rx module have basically 5 stages

1. IDLE stage  
it waits for the rx signal to go low. When the rx signal goes low the data transfer actually begins. So the state will goes to START stage when the rx goes low
2. START stage  
in this stage since we have the tick pulse 16 times per bit. I looked at the 7<sup>th</sup> bit if that is actually 1 or 0. then if it is 0 go to the data stage or else go to the idle state since it might be a glitch. Only transition to the next state once the rx\_tick\_cnt=15 since we are generating tick 16 times right
3. DATA stage  
in this stage create a counter to count for 8 data bits and for each data bits we sample when tick is from 6 to 10 and count for the majority vote. If majority value is 1 that is known by adding a vairable by 1 everytime the value is 1. thus if vairable is = or greater than 3 the data is 1 or else it is 0. then shift the data to a data reg.
4. STOP stage  
after the data is recieved it moves to stop stage in that we smaple the 7<sup>th</sup> bit if it is still low then it is error and moves to error stage else moves to idle stage making the ready\_signal 1. that is bit recieved successfully for 1 cycle.
5. ERROR stage.  
We move to the idle stage only when the rx goes to high and tick cnt counts till 15

## Verilog Code

```
module rx (  
    input clk,  
    input rstn,  
    input rx_tick,  
    input read_clear,  
    input rx,  
    output ready,  
    output [7:0] data_out  
);  
  
localparam [2:0]  
    RX_IDLE = 3'd0,  
    RX_START = 3'd1,  
    RX_DATA = 3'd2,  
    RX_STOP = 3'd3,  
    RX_ERROR = 3'd4;  
  
reg start_valid = 1'b0;  
reg stop_valid = 1'b0;  
reg [2:0] rx_state = RX_IDLE;  
reg [3:0] rx_tick_cnt = 4'd0;  
reg rx_sync_1 = 1'b1;  
reg rx_sync_2 = 1'b1;  
  
reg [2:0] sampling = 3'd0;  
reg [3:0] data_cnt = 4'd0;  
reg [7:0] data = 8'd0;
```

```

reg ready_reg = 1'b0;

assign ready = ready_reg;
assign data_out = data;

//to prevent metastability
always @(posedge clk) begin
    rx_sync_1 <= rx;
    rx_sync_2 <= rx_sync_1;
end

// Main FSM
always @(posedge clk) begin
    if (!rstn) begin
        rx_state  <= RX_IDLE;
        rx_tick_cnt <= 4'd0;
        data_cnt  <= 4'd0;
        sampling  <= 3'd0;
        start_valid <= 1'b0;
        stop_valid <= 1'b0;
        ready_reg  <= 1'b0;
    end else begin

        if (read_clear) begin
            ready_reg <= 1'b0;
        end

        case (rx_state)

            RX_IDLE: begin
                start_valid <= 1'b0;
                stop_valid  <= 1'b0;
                rx_tick_cnt <= 4'd0;
                data_cnt    <= 4'd0;
                if (!rx_sync_2) begin
                    rx_state <= RX_START;
                end
            end

            RX_START: begin
                if (rx_tick) begin
                    rx_tick_cnt <= rx_tick_cnt + 4'd1;

                    if (rx_tick_cnt == 4'd7) begin
                        if (!rx_sync_2) begin
                            start_valid <= 1'b1;
                        end
                    end
                end
            end

            if (rx_tick_cnt == 4'd15) begin
                if (start_valid) begin
                    rx_state  <= RX_DATA;
                end
            end
        end
    end
end

```

```

    sampling    <= 3'd0;
    data_cnt    <= 4'd0;
    rx_tick_cnt <= 4'd0;
end else begin
    rx_state    <= RX_IDLE;
end
end
end

    end
end

RX_DATA: begin
    if (rx_tick) begin
        rx_tick_cnt <= rx_tick_cnt + 4'd1;
        if (rx_tick_cnt <= 4'd5) sampling <= 3'd0;

        if (rx_tick_cnt == 4'd6) sampling <= sampling + rx_sync_2;
        if (rx_tick_cnt == 4'd7) sampling <= sampling + rx_sync_2;
        if (rx_tick_cnt == 4'd8) sampling <= sampling + rx_sync_2;
        if (rx_tick_cnt == 4'd9) sampling <= sampling + rx_sync_2;
        if (rx_tick_cnt == 4'd10) sampling <= sampling + rx_sync_2;

        if (rx_tick_cnt == 4'd15) begin

            if (sampling >= 3'd3) begin
                data <= {1'b1, data[7:1]};
            end else begin
                data <= {1'b0, data[7:1]};
            end

            data_cnt <= data_cnt + 4'd1;
            sampling <= 3'd0;

            if (data_cnt == 4'd7) begin
                rx_state <= RX_STOP;
            end
        end
    end
end
end

RX_STOP: begin
    if (rx_tick) begin
        rx_tick_cnt <= rx_tick_cnt + 4'd1;

        if (rx_tick_cnt == 4'd7) begin
            if (rx_sync_2) begin
                stop_valid <= 1'b1;
            end
        end

        if (rx_tick_cnt == 4'd15) begin
            if (stop_valid) begin

```

```

        rx_state <= RX_IDLE;
        ready_reg <= 1'b1;
    end else begin
        rx_state <= RX_ERROR;
    end
end
end
end
end

RX_ERROR: begin
    if (rx_tick) begin
        rx_tick_cnt <= rx_tick_cnt + 4'd1;

        if (rx_tick_cnt == 4'd15 && rx_sync_2) begin
            rx_state <= RX_IDLE;
        end
    end
end
end

    default: rx_state <= RX_IDLE;

endcase
end
end

endmodule

```

## TX Module

it consist of 4 stages IDLE,START,DATA,STOP

1. IDLE  
it is sending the tx high pulse. Stay at this stage till the enable pulse comes.
2. START  
start sending the start pulse. Count for 16 pulses and send the low state for that. That is the start stage
3. DATA state  
for each data bit count till 15 and using shift register shift the data to send and using a counter send the 8 bit data one by one.
4. STOP state  
this state is the high state for 15 ticks.

```

module tx (
    input clk,
    input rstn,
    input tx_tick,
    input enable,
    input [7:0] data,
    output tx,
    output busy
);

    reg tx_reg = 1'b1;
    assign tx = tx_reg;

```

```
assign busy = (tx_state != TX_IDLE);
```

```
localparam [1:0]  
    TX_IDLE = 2'd0,  
    TX_START = 2'd1,  
    TX_DATA = 2'd2,  
    TX_STOP = 2'd3;
```

```
reg [1:0] tx_state = TX_IDLE;  
reg [7:0] tx_data_reg = 8'd0;  
reg [2:0] bit_cnt = 3'd0;  
reg [3:0] tick_cnt = 4'd0;
```

```
reg tx_sync_1 = 1'b0;  
reg tx_sync_2 = 1'b0;
```

```
always @(posedge clk) begin  
    tx_sync_1 <= enable;  
    tx_sync_2 <= tx_sync_1;  
end
```

```
always @(posedge clk) begin  
    if (!rstn) begin  
        tx_state <= TX_IDLE;  
        tx_reg <= 1'b1;  
        tx_data_reg <= 8'd0;  
        bit_cnt <= 3'd0;  
        tick_cnt <= 4'd0;  
    end else begin  
        case (tx_state)  
  
            TX_IDLE: begin  
                tx_reg <= 1'b1;  
                tick_cnt <= 4'd0;  
                bit_cnt <= 3'd0;  
                if (tx_sync_2) begin  
                    tx_state <= TX_START;  
                    tx_data_reg <= data;  
                end  
            end  
  
            TX_START: begin  
                tx_reg <= 1'b0;  
                if (tx_tick) begin  
                    if (tick_cnt == 4'd15) begin  
                        tick_cnt <= 4'd0;  
                        tx_state <= TX_DATA;  
                    end else begin  
                        tick_cnt <= tick_cnt + 4'd1;  
                    end  
                end  
            end  
  
            TX_DATA: begin  
                tx_reg <= 1'b1;  
                if (tx_sync_2) begin  
                    tx_state <= TX_STOP;  
                    tx_data_reg <= data;  
                end  
            end  
  
            TX_STOP: begin  
                tx_reg <= 1'b0;  
                if (tx_tick) begin  
                    if (tick_cnt == 4'd15) begin  
                        tick_cnt <= 4'd0;  
                        tx_state <= TX_IDLE;  
                    end else begin  
                        tick_cnt <= tick_cnt + 4'd1;  
                    end  
                end  
            end  
  
            default: begin  
                tx_reg <= 1'b1;  
                tick_cnt <= 4'd0;  
                bit_cnt <= 3'd0;  
                tx_state <= TX_IDLE;  
            end  
        endcase  
    end  
end
```

```

TX_DATA: begin
    tx_reg <= tx_data_reg[0];
    if (tx_tick) begin
        if (tick_cnt == 4'd15) begin
            tick_cnt <= 4'd0;
            tx_data_reg <= tx_data_reg >> 1;
            if (bit_cnt == 3'd7) begin
                bit_cnt <= 3'd0;
                tx_state <= TX_STOP;
            end else begin
                bit_cnt <= bit_cnt + 3'd1;
            end
        end else begin
            tick_cnt <= tick_cnt + 4'b1;
        end
    end
end

TX_STOP: begin
    tx_reg <= 1'b1;
    if (tx_tick) begin
        if (tick_cnt == 4'd15) begin
            tick_cnt <= 4'd0;
            tx_state <= TX_IDLE;
        end else begin
            tick_cnt <= tick_cnt + 4'd1;
        end
    end
end

default: tx_state <= TX_IDLE;

endcase
end
end

```

endmodule

## UART TOP MODULE

created by just connecting those pins of each module  
module uart(

input clk,  
input rstn,

input tx\_enable,  
input [7:0] data,  
output tx,  
output busy,

```
input read_clear,  
input rx,  
output ready,  
output [7:0] data_out
```

```
);
```

```
wire tick;
```

```
baud_generator bg(
```

```
.clk(clk),  
.rstn(rstn),  
.tick(tick)
```

```
);
```

```
tx tx1(
```

```
.clk(clk),  
.rstn(rstn),  
.tx_tick(tick),  
.enable(tx_enable),  
.data(data),  
.tx(tx),  
.busy(busy)
```

```
);
```

```
rx rx1(
```

```
.clk(clk),  
.rstn(rstn),  
.rx_tick(tick),  
.read_clear(read_clear),  
.rx(rx),  
.ready(ready),  
.data_out(data_out)
```

```
);
```

```
endmodule
```

TESTED the uart moduel by connecting the tx pin to rx pin and verify if the transmitteed byte is displaying or not.

```
import cocotb
```

```
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge, ClockCycles
```

```
@cocotb.test()
async def test_uart_loopback(dut):
```

```
    cocotb.start_soon(Clock(dut.clk, 83.33, unit="ns").start())
```

```
    dut.rstn.value=0
    dut.tx_enable.value=0
    dut.data.value=0
    dut.read_clear.value=0
    dut.rx.value=1
```

```
    await ClockCycles(dut.clk,5)
    dut.rstn.value=1
    await ClockCycles(dut.clk,5)
```

```
    byte=0x10
    dut.data.value=byte
    dut.tx_enable.value=1
```

```
    await RisingEdge(dut.clk)
    dut.tx_enable.value=0
    dut._log.info(f"tx byte {byte}")
    timeout_counter=0
    max_cnt=2000
```

```
    while int(dut.ready.value)==0:
        await RisingEdge(dut.clk)

        dut.rx.value = int(dut.tx.value)
        timeout_counter+=1
        assert timeout_counter<max_cnt, "timeout occurred"
```

```
    recieved_data=int(dut.data_out.value)
    dut._log.info(f"recieved {recieved_data}")
```

```
rm -f results.xml
"make" -f makefile results.xml
make[1]: Entering directory '/home/kevin/Desktop/uart_streamed_fir_filter/verilog/uart'
rm -f results.xml
MODULE=tb_uart TESTCASE= TOPLEVEL=uart TOPLEVEL_LANG=verilog \
/home/kevin/fpga/oss-cad-suite/bin/vvp -M /home/kevin/fpga/oss-cad-suite/lib/python3.11/site-packages/cocotb-2.0.0.dev0+393b3284-py3.1
sim_build/sim.vvp 2>&1 | tee sim_build/sim.log
y3.11-linux-x64.egg/cocotb
0.00ns INFO gpi ..mbed/gpi_embed.cpp:108 in set_program_name_in_venv Using Python virtual envir
0.00ns INFO gpi ../gpi/GpiCommon.cpp:101 in gpi_print_registered_impl VPI registered
0.00ns INFO cocotb Running on Icarus Verilog version 13.0 (devel)
0.00ns INFO cocotb Running tests with cocotb v2.0.0.dev0+393b3284 from /home/kevin/fpga/oss-cad-suite/lib/
0.00ns INFO cocotb Seeding Python random module with 1790142927
0.00ns INFO cocotb.regression running tb_uart.test_uart_loopback (1/1)
833.30ns INFO cocotb.uart tx byte 16
87913.15ns INFO cocotb.uart recieved 16
87913.15ns INFO cocotb.regression tb_uart.test_uart_loopback passed
87913.15ns INFO cocotb.regression
*****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** tb_uart.test_uart_loopback PASS 87913.15 0.16 545819.67 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0 87913.15 0.17 531323.05 **
*****
```

here verified send 16 and recieved bit 16 too.

## **FILTER TOP MODULE**

before that i added a enable in filter module. Because i need to make the filter process only when i needed not like on a clock edge.

```
module uart_with_filter (  
    input wire clk,  
    input wire rx,  
    output wire tx,  
    input wire btn1_n  
);  
  
    wire rx_ready;  
    wire [7:0] rx_byte;  
    wire [7:0] filtered_byte;  
    wire tx_busy;  
  
    reg tx_enable = 1'b0;  
    reg read_clear = 1'b0;  
    reg [7:0] tx_byte = 8'd0;  
  
    reg filter_en = 1'b0;  
    reg pending_tx = 1'b0;  
  
    reg rst_n_sync1, rst_n;  
    always @(posedge clk) begin  
        rst_n_sync1 <= btn1_n;  
        rst_n <= rst_n_sync1;  
    end  
  
    uart uart0 (  
        .clk(clk),  
        .rstn(rst_n),  
        .tx_enable(tx_enable),  
        .data(tx_byte),  
        .tx(tx),  
        .busy(tx_busy),  
        .read_clear(read_clear),  
        .rx(rx),  
        .ready(rx_ready),  
        .data_out(rx_byte)  
    );  
  
    filter filter0 (  
        .clk(clk),  
        .rstn(rst_n),  
        .en(filter_en),  
        .sig(rx_byte),  
        .samp_op(),  
        .trunc_op(filtered_byte)  
    );
```

```

always @(posedge clk)
if (!rst_n) begin
    tx_enable <= 1'b0;
    read_clear <= 1'b0;
    filter_en <= 1'b0;
    pending_tx <= 1'b0;
    tx_byte <= 8'd0;
end else
begin
    tx_enable <= 1'b0;
    read_clear <= 1'b0;
    filter_en <= 1'b0;

    if (rx_ready && !tx_busy && !pending_tx) begin
        filter_en <= 1'b1;
        read_clear <= 1'b1;
        pending_tx <= 1'b1;
    end

    else if (pending_tx) begin
        tx_byte <= filtered_byte;
        tx_enable <= 1'b1;
        pending_tx <= 1'b0;
    end
end
end

```

endmodule

i programmed the board with that and send the data and revied using python serial communication

```

import serial
import time
ser = serial.Serial('/dev/ttyUSB1', 115200, timeout=1)
time.sleep(0.1)

```

```

X = [60,70,80,90,100]
ser.write(bytes(X))
response = ser.read(len(X))
received_dec = [b for b in response]
print(f"Sent: {X}")
print(f"Received : {received_dec}")

```

```

print(len(received_dec))
print(len(X))

```

here timeout means it would wait atmost 1sec for data

programed the baord using apio build and upload then the output of the python is

```

[Wed Sep 23 11:31:53 2026] Processing icePie
-----
yosys -p "synth_ice40 -top uart_with_filter -json hardware.json" -q baud_generator.v fir.v rx.v tx.v uart.v uart_with_filter.v
Warning: Replacing memory \signal with list of registers. See fir.v:42, fir.v:34
nextpnr-ice40 --up5k --package sg48 --json hardware.json --asc hardware.asc --pcf icepie.pcf -q
[SUCCESS] Took 6.85 seconds
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/top_module$ apio upload
[Wed Sep 23 11:32:03 2026] Processing icePie
-----
iceprog -d i:0x0403:0x6010:0 hardware.bin
init...
cdone: high
reset...
cdone: low
flash ID: 0xEF 0x40 0x18 0x00
file size: 104090
erase 64kB sector at 0x000000..
erase 64kB sector at 0x010000..
done.
VERIFY OK
cdone: high
Bye.
-----
[SUCCESS] Took 4.39 seconds
-----
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/top_module$ python3
abc.history      fir.v          hardware.json   _pycache_/      .sconsign.dblite  sim            tx.v
apio.ini         hardware.asc    icepie.pcf      results.xml     serialsend.py     sim_build/     uart.v
baud_generator.v hardware.bin     makefile        rx.v           signal_gen.py     tb_uart.py     uart_with_filter.v
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/top_module$ python3 serialsend.py
Sent: [60, 70, 80, 90, 100]
Received : [0, 0, 2, 5, 3]
5

```

which is verified using python convolution function and truncating

```

(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/fir_filter$ iverilog -o sim tb_index.v index.v
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/fir_filter$ vvp sim
VCD info: dumpfile tb_index.vcd opened for output.
cycle =      1  sig = 60  samp_op =      0  trunc_op =      0
cycle =      2  sig = 70  samp_op =      0  trunc_op =      0
cycle =      3  sig = 80  samp_op =    300  trunc_op =      2
cycle =      4  sig = 90  samp_op =    590  trunc_op =      5
cycle =      5  sig = 100 samp_op =    380  trunc_op =      3

```

this is output from the fir filter last week.